

LAPORAN TUGAS KECIL 3

Algoritma Pathfinding untuk Pencarian Solusi Ice Sliding Puzzle

IF2211 STRATEGI ALGORITMA
SEMESTER II Tahun 2025/2026



Ramadhian Nabil Firdaus Gumay / 13524126
Leonardus Brain Fatolosja / 13524146

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Leonardus Brain Fatolosja

Ramadhian Nabil Firdaus Gumay

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

I Penjelasan dan Analisis Algoritma Pathfinding

Pada program ini, terdapat lima algoritma yang diimplementasikan, yaitu Algoritma Wajib (UCS, GBFS, dan A*) dan Algoritma Bonus (DFS dan BFS). Untuk heuristik, terdapat total tiga heuristik yang diimplementasikan di dalam GBFS dan A* untuk menghitung perkiraan cost sampai target. Ketiga heuristiknya di antara lain:

- Heuristik Straight Line Distance

Heuristik pertama adalah mengukur estimasi jarak dari posisi sekarang ke target dengan menggunakan pengukuran garis lurus. Rumus yang digunakan adalah

$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. Pada Heuristik pertama, target yang dihitung adalah target selanjutnya saja (tidak selalu target akhir).

- Heuristik Manhattan Distance

Heuristik kedua adalah mengukur estimasi jarak dengan menggunakan pendekatan manhattan distance. Dibandingkan mengukur jarak dengan garis lurus, manhattan menggunakan garis yang berbelok-belok sehingga diharapkan lebih akurat dibandingkan SLD. Rumus yang digunakan adalah $|x_2 - x_1| + |y_2 - y_1|$. Pada heuristik kedua, target yang dihitung adalah target selanjutnya saja (tidak selalu target akhir).

- Heuristik Manhattan Distance dengan Banyak Target

Heuristik ketiga adalah mengukur estimasi jarak dengan menggunakan pendekatan manhattan distance yang berbeda dibandingkan versi kedua. Dibandingkan mengukur jarak hanya dengan satu target, versi ini menghitung banyak target sampai target terakhir untuk mendapatkan estimasi yang lebih mendekati hasil. Rumus yang digunakan adalah $|x_2 - x_1| + |y_2 - y_1|$. Pada heuristik kedua, target yang dihitung adalah semua target selanjutnya sampai target terakhir.

Setelah mengetahui heuristik yang dipakai, penjelasan dan analisis masing-masing algoritma ditulis pada penjelasan di bawah ini. Sebagai catatan, **iterasi** yang dihitung di dalam program adalah setiap kali sebuah elemen ditelusuri dalam artian setiap elemen dipilih di dalam priority queue atau queue biasa.

A. Algoritma UCS

Algoritma *Uniform Cost Search* (UCS) merupakan salah satu metode pencarian yang bertujuan untuk menemukan jalur dengan total biaya (*cost*) paling minimum dari simpul awal ke simpul tujuan. Berbeda dengan pencarian seperti BFS yang hanya berfokus pada penemuan solusi dengan jumlah langkah paling sedikit, algoritma UCS secara khusus memperhitungkan bobot atau jarak yang ada pada setiap sisi antar simpul di dalam graf.

Dalam mekanisme kerjanya, algoritma ini menggunakan fungsi $g(n)$, yang merepresentasikan total akumulasi biaya lintasan dari simpul akar (*root*) hingga ke simpul n . Pencarian dilakukan dengan cara selalu mengekspansi simpul hidup (*live nodes*) yang memiliki nilai $g(n)$ paling kecil terlebih dahulu. Simpul-simpul yang menunggu untuk dieksplorasi ini biasanya dikelola dalam sebuah antrean berprioritas, di mana simpul dengan biaya terendah selalu mendapat giliran pertama untuk diproses. Proses ekspansi ini akan terus berjalan hingga algoritma mencapai

simpul tujuan. Karena sifatnya yang selalu memprioritaskan rute termurah untuk dievaluasi, algoritma UCS dijamin optimal untuk menemukan solusi terbaik berdasarkan total biaya lintasan.

```
ALGORITMA UCS (map)

Buat priorityQueue
Buat visited[baris] [kolom] [targetAngka]

startNode <- StateNode (
  posisi awal Z,
  totalCost = 0,
  targetAngka = 0,
  isGameOver = false,
  path = ""
)

Masukkan startNode ke priorityQueue
WHILE (priorityQueue tidak kosong) :

  current <- ambil state dengan totalCost terkecil

  IF visited[current.r] [current.c] [current.targetAngka] = true:
    continue

  visited[current.r] [current.c] [current.targetAngka] = true

  IF current berada di posisi goal O AND current.targetAngka sudah melewati semua angka:
    return current sebagai solusi

  FOR setiap arah dalam [atas, kanan, bawah, kiri]:
    nextNode <- slideMove (map, current, arah)

    IF nextNode != null AND nextNode bukan game over:
      Masukkan nextNode ke priorityQueue

return null
```

B. Algoritma GBFS

Algoritma Greedy Best First Search merupakan metode pencarian informed yang berarti menggunakan informasi cost yang tersedia untuk menentukan jalur menuju solusi. Algoritma ini menggunakan fungsi heuristik untuk menentukan simpul terbaik untuk ditelusuri. Pada program ini, setiap state berisi posisi aktor, total cost sementara, target angka berikutnya, status game over, dan path gerakan. GBFS menyimpan state dalam priority queue berdasarkan nilai heuristik $h(n)$, lalu selalu mengambil state dengan estimasi jarak terkecil menuju target berikutnya atau goal. Setelah sebuah state diambil, program mencoba empat arah gerakan, yaitu atas, kanan, bawah, dan kiri, menggunakan aturan ice sliding sampai aktor berhenti, terkena lava, keluar papan, atau melanggar urutan angka. Pada tiap percobaan arah gerakan, state akhir dari gerakan tersebut akan dimasukkan ke dalam priority queue yang ditentukan berdasarkan estimasi cost ke state target.

```

ALGORITMA GBFS(map, heuristicOption)

Buat priorityQueue
Buat visited[baris][kolom][targetAngka]

startNode <- StateNode(
    posisi awal Z,
    totalCost = 0,
    targetAngka = 0,
    isGameOver = false,
    path = ""
)

startNode.estimatedTotalCost <- heuristic(startNode, heuristicOption)
Masukkan startNode ke priorityQueue

WHILE (priorityQueue tidak kosong):
    current <- ambil state dengan estimatedTotalCost terkecil

    IF visited[current.r][current.c][current.targetAngka] = true:
        continue

    visited[current.r][current.c][current.targetAngka] = true

    IF current berada di posisi goal 0 AND current.targetAngka sudah melewati semua angka:
        return current sebagai solusi

    FOR setiap arah dalam [atas, kanan, bawah, kiri]:
        nextNode <- slideMove(map, current, arah)

        IF nextNode != null AND nextNode bukan game over:
            nextNode.estimatedTotalCost <- heuristic(nextNode, heuristicOption)
            Masukkan nextNode ke priorityQueue

return null

```

Dalam GBFS, nilai prioritas yang dipakai adalah $f(n) = h(n)$, dimana $h(n)$ adalah estimasi cost dari posisi tersebut sampai posisi tujuan sehingga cost aktual dari start tidak menjadi pertimbangan saat memilih state berikutnya. Sementara itu, $g(n)$ tetap tersedia sebagai total cost yang sudah ditempuh dari posisi awal sampai state tersebut, tetapi tidak digunakan untuk menentukan prioritas pada GBFS. Karena sifatnya greedy, GBFS biasanya dapat menemukan solusi lebih cepat, tetapi tidak menjamin solusi dengan cost minimum. Heuristik yang disediakan untuk menghitung $h(n)$ ada 3, yaitu heuristik SLD, Manhattan Distance (Satu Target), dan Manhattan Distance (Banyak Target).

C. Algoritma A*

A* juga merupakan algoritma informed search, tetapi lebih seimbang karena mempertimbangkan cost aktual dan estimasi sisa jarak. Algoritma A* pada program ini bekerja dengan menyimpan state dalam priority queue berdasarkan nilai total cost, yaitu $f(n) = g(n) + h(n)$. State awal dibuat dari posisi Z dengan $totalCost = 0$, $targetAngka = 0$, path kosong, lalu nilai estimasinya dihitung menggunakan heuristik yang dipilih user. Selama priority queue belum kosong, program mengambil state dengan nilai $f(n)$ terkecil, melewati state yang sudah pernah dikunjungi, lalu menandai state tersebut sebagai visited berdasarkan posisi baris, kolom, dan target angka berikutnya. Jika state tersebut sudah berada tepat di goal 0 dan semua angka sudah dilewati secara berurutan, maka state dikembalikan sebagai solusi. Jika belum, program mencoba empat arah gerakan, yaitu atas, kanan, bawah, dan kiri, menggunakan fungsi pergerakan dengan mengikuti aturan ice sliding. Setiap hasil gerakan yang valid dan tidak game over akan dihitung ulang nilai $f(n)$ -nya dari total cost aktual ditambah estimasi heuristik menuju target berikutnya

atau goal, kemudian dimasukkan kembali ke priority queue untuk diproses pada iterasi berikutnya.

```

ALGORITMA A*(map, heuristicOption)

Buat priorityQueue
Buat visited[baris][kolom][targetAngka]

startNode <- StateNode(
    posisi awal 2,
    totalCost = 0,
    targetAngka = 0,
    isGameOver = false,
    path = ""
)

startNode.estimatedTotalCost <- startNode.totalCost + heuristic(startNode, heuristicOption)

Masukkan startNode ke priorityQueue

WHILE priorityQueue tidak kosong:
    current <- ambil state dengan estimatedTotalCost terkecil

    IF visited[current.r][current.c][current.targetAngka] = true:
        continue

    visited[current.r][current.c][current.targetAngka] = true

    IF current berada di posisi goal 0 AND current.targetAngka sudah melewati semua angka:
        return current sebagai solusi

    FOR setiap arah dalam [atas, kanan, bawah, kiri]:
        nextNode <- slideMove(map, current, arah)
        IF nextNode != null AND nextNode bukan game over:
            g(n) <- nextNode.totalCost
            h(n) <- heuristic(nextNode, heuristicOption)
            nextNode.estimatedTotalCost <- g(n) + h(n)

            Masukkan nextNode ke priorityQueue

return null

```

Dalam A*, nilai prioritas yang dipakai adalah $f(n) = h(n) + g(n)$, dimana $h(n)$ adalah estimasi cost dari posisi tersebut sampai posisi tujuan dan $g(n)$ adalah cost aktual dari posisi awal ke posisi tersebut sehingga algoritma A* lebih menjamin solusi terbaik dibandingkan solusi yang lain. Heuristik yang disediakan untuk menghitung $h(n)$ ada 3, yaitu heuristik SLD, Manhattan Distance (Satu Target), dan Manhattan Distance (Banyak Target).

D. Algoritma BFS

Algoritma *Breadth-First Search* (BFS) merupakan salah satu jenis algoritma pencarian buta (*uninformed search*) yang melakukan penelusuran graf secara melebar. Pencarian dengan BFS dimulai dari simpul awal dan mengunjungi seluruh simpul yang bertetangga langsung dengan simpul tersebut terlebih dahulu. Setelah semua tetangga di kedalaman pertama selesai dikunjungi, algoritma baru akan beralih mengekskansi tetangga dari simpul-simpul tersebut. Proses ini terus berlanjut secara bertahap lapis demi lapis (*level by level*) hingga simpul tujuan (*goal*) ditemukan. Dalam implementasinya, BFS menggunakan struktur data antrian (*queue*) dengan prinsip *First-In-First-Out* (FIFO) untuk menyimpan status simpul hidup yang menunggu giliran untuk diekspansi. Selain antrian, BFS juga membutuhkan matriks ketetanggaan serta *array boolean* (tabel penanda status kunjungan) untuk mencatat simpul-simpul mana saja yang sudah dieksplorasi. Hal ini penting agar pencarian tidak terjebak pada perulangan kunjungan simpul

yang sama. Karakteristik BFS yang mengekskansi ruang status secara merata membuatnya dijamin (*complete*) bisa menemukan solusi selama percabangannya terbatas, serta optimal dalam menemukan jalur dengan jumlah langkah yang paling sedikit.

```
ALGORITMA BFS (map)
Buat queue
Buat visited[baris] [kolom] [targetAngka]

startNode <- StateNode (
  posisi awal Z,
  totalCost = 0,
  targetAngka = 0,
  isGameOver = false,
  path = ""
)

Masukkan startNode ke queue
WHILE (queue tidak kosong) :

  current <- ambil state pertama dari queue

  IF visited[current.r] [current.c] [current.targetAngka] = true:
    continue

  visited[current.r] [current.c] [current.targetAngka] = true

  IF current berada di posisi goal O AND current.targetAngka sudah melewati semua angka:
    return current sebagai solusi

  FOR setiap arah dalam [atas, kanan, bawah, kiri]:
    nextNode <- slideMove (map, current, arah)

    IF nextNode != null AND nextNode bukan game over:
      Masukkan nextNode ke queue

return null
```

E. Algoritma DFS

Algoritma *Depth-First Search* (DFS) merupakan metode pencarian yang melakukan penelusuran graf secara mendalam. Berkebalikan dengan BFS yang mengekskansi graf lapis demi lapis, DFS berfokus untuk terus bergerak mengekskansi cabang menuju simpul anak terdalam terlebih dahulu. Dalam implementasinya, algoritma DFS memanfaatkan struktur data tumpukan (*stack*) yang beroperasi dengan prinsip *Last-In-First-Out* (LIFO) untuk mengelola agenda simpul-simpul hidup yang menunggu antrean untuk dieksplorasi.

Cara kerjanya adalah pencarian akan terus bergerak menelusuri satu cabang secara vertikal ke bawah sampai menemukan simpul tujuan atau hingga menemui jalan buntu (simpul daun yang tidak lagi memiliki anak/tetangga baru). Jika menemui jalan buntu, algoritma akan melakukan proses runut-balik (*backtracking*) ke simpul induk terdekat untuk mengeksplorasi cabang lain yang belum sempat dikunjungi. Keuntungan dari DFS adalah penggunaan memorinya yang relatif kecil karena hanya perlu menyimpan status lintasan yang sedang dievaluasi saat itu. Namun, kelemahan utama dari DFS adalah pencarian ini tidak dijamin optimal; jika solusi ditemukan, solusi tersebut belum tentu merupakan lintasan terpendek atau yang memiliki biaya terkecil.

```

ALGORITMA DFS (map)
Buat stack
Buat visited[baris] [kolom] [targetAngka]

startNode <- StateNode (
  posisi awal Z,
  totalCost = 0,
  targetAngka = 0,
  isGameOver = false,
  path = ""
)

Masukkan startNode ke stack
WHILE (stack tidak kosong) :

  current <- ambil state terakhir dari stack

  IF visited[current.r] [current.c] [current.targetAngka] = true:
    continue

  visited[current.r] [current.c] [current.targetAngka] = true

  IF current berada di posisi goal O AND current.targetAngka sudah melewati semua angka:
    return current sebagai solusi

  FOR setiap arah dalam [atas, kanan, bawah, kiri]:
    nextNode <- slideMove (map, current, arah)

    IF nextNode != null AND nextNode bukan game over:
      Masukkan nextNode ke stack

return null

```

F. Penjelasan Heuristik

Terdapat tiga heuristik pada program ini. Heuristik tersebut digunakan untuk menghitung estimasi jarak dari posisi saat itu ke posisi target. Heuristik ini secara khusus digunakan pada algoritma GBFS (Greedy Best First Search) dan A*. Masing-masing heuristik dapat dijelaskan sebagai berikut.

1. Heuristik Straight Line Distance

Perhitungan estimasi heuristik SLD menggunakan rumus untuk menghitung garis lurus. Dalam algoritma A*, SLD cenderung **admissible** jika cost minimum setiap tile bernilai minimal 1, karena jarak garis lurus biasanya tidak akan melebihi jarak lintasan sebenarnya. Namun, karena permainan menggunakan mekanisme ice sliding dan cost tile berbeda-beda, admissibility tetap bergantung pada konfigurasi map dan nilai cost yang diberikan.

2. Heuristik Manhattan Distance

Heuristik Manhattan Distance menghitung estimasi jarak berdasarkan selisih baris dan kolom antara posisi aktor saat ini dengan target berikutnya. Heuristik ini cocok untuk papan grid karena pergerakan dilakukan secara horizontal dan vertikal. Pada program, Manhattan Distance digunakan untuk memperkirakan jarak menuju angka berikutnya atau goal. Untuk A*, heuristik ini bisa **admissible** jika setiap perpindahan antar tile

memiliki cost minimal 1 dan tidak ada kondisi yang membuat estimasi melebihi cost sebenarnya. Namun, pada ice sliding puzzle, aktor tidak bergerak satu langkah per tile secara bebas, sehingga Manhattan Distance tidak selalu merepresentasikan cost traversal sebenarnya secara sempurna tetapi tetap memberikan estimasi yang admissible karena tidak akan overestimate.

3. Heuristik Manhattan Distance (Banyak Target)

Sama seperti sebelumnya, akan tetapi heuristik ini menghitung estimasi jarak dari posisi aktor saat ini ke seluruh target yang tersisa secara berurutan, lalu dilanjutkan sampai titik tujuan O. Misalnya jika aktor masih harus melewati angka 1, 2, lalu goal, maka heuristik akan menjumlahkan Manhattan Distance dari posisi sekarang ke 1, dari 1 ke 2, dan dari 2 ke O. Heuristik ini memberi estimasi yang lebih informatif dibanding Manhattan Distance biasa karena mempertimbangkan urutan angka yang wajib dilewati. Namun, untuk A*, heuristik ini **belum tentu admissible** pada semua kasus karena hasil penjumlahan jarak geometris bisa saja tidak sesuai dengan cost aktual akibat sliding, rintangan, lava, dan variasi cost tile. Oleh karena itu, heuristik ini bisa menjadi lebih akurat dibandingkan yang lain namun dengan resiko menjadi tidak admissible.

G. Perbandingan Algoritma

Secara teoritis, UCS menjamin solusi dengan cost minimum karena selalu memilih state dengan total cost terkecil dari awal, yaitu berdasarkan $g(n)$. Pada permasalahan Ice Sliding Puzzle, hal ini penting karena setiap tile memiliki cost traversal yang berbeda, sehingga jalur dengan jumlah langkah lebih sedikit belum tentu memiliki cost total paling kecil. Namun, UCS cenderung mengeksplorasi lebih banyak konfigurasi karena tidak memiliki informasi arah menuju goal. GBFS sebaliknya lebih cepat dalam banyak kasus karena hanya mempertimbangkan heuristik $h(n)$ untuk memilih state yang terlihat paling dekat dengan target, tetapi tidak menjamin solusi optimal karena mengabaikan cost aktual yang sudah ditempuh.

A* berada di tengah antara UCS dan GBFS karena menggunakan $f(n) = g(n) + h(n)$, yaitu gabungan cost aktual dari start dan estimasi cost menuju target. Jika heuristik yang digunakan admissible, A* dapat menemukan solusi optimal dengan jumlah eksplorasi yang biasanya lebih sedikit dibanding UCS. Pada implementasi ini juga terdapat BFS dan DFS sebagai algoritma bonus. BFS mencari berdasarkan kedalaman/jumlah gerakan sehingga tidak mempertimbangkan cost tile dan tidak menjamin cost minimum, sedangkan DFS cenderung hemat memori tetapi sangat bergantung pada urutan eksplorasi dan tidak menjamin solusi optimal maupun solusi tercepat.

Code

Setelah penjelasan masing-masing algoritma, maka implementasi dalam bahasa pemrograman **Java** pun bisa dilakukan. Code yang ditampilkan merupakan kode-kode yang berhubungan dengan penjelasan di atas (hanya kode algoritma dan heuristik).


```

public class Astar{

    public StateNode search(GameMap map, int optionHeuristic){
        iterationCount = 0;
        Heuristic heuristic = new Heuristic(map);
        PriorityQueue<StateNode> pq = new PriorityQueue<>();
        boolean[][][] visited = new boolean[map.rows][map.cols][map.totalAngka + 1];

        StateNode node = new StateNode(map.startR, map.startC, totalCost: 0, targetAngka: 0, isGameOver: false, parent: null);
        node.estimatedTotalCost = node.totalCost + heuristic.estimateCost(node.r, node.c, node.targetAngka, optionHeuristic);

        pq.add(node);

        while (!pq.isEmpty()){
            StateNode current = pq.poll();

            if (visited[current.r][current.c][current.targetAngka]){
                continue;
            }

            iterationCount++;
            visited[current.r][current.c][current.targetAngka] = true;

            if (current.r == map.goalR && current.c == map.goalC && current.targetAngka == map.totalAngka) {
                return current;
            }

            for (int i=0;i<4;i++){
                StateNode newNode = IceSlidingLogic.slideMove(map.grid, map.costMatrix, current, i);

                if (newNode != null && !(newNode.isGameOver)){
                    // f(n) = g(n) + h(n)
                    newNode.estimatedTotalCost = newNode.totalCost + heuristic.estimateCost(newNode.r, newNode.c, newNode.targetAngka, optionHeuristic);
                    pq.add(newNode);
                }
            }
        }
    }
}

```

Gambar 1. Algoritma A*

```

public class BFS {
    private int iterationCount;

    public StateNode search(GameMap map) {
        iterationCount = 0;
        Queue<StateNode> q = new LinkedList<>();
        boolean[][][] visited = new boolean[map.rows][map.cols][map.totalAngka + 1];

        StateNode startNode = new StateNode(map.startR, map.startC, totalCost: 0, targetAngka: 0, isGameOver: false, parent: null);
        q.add(startNode);

        while (!q.isEmpty()) {
            StateNode current = q.poll();

            if (visited[current.r][current.c][current.targetAngka]) {
                continue;
            }

            iterationCount++;
            visited[current.r][current.c][current.targetAngka] = true;

            if (current.r == map.goalR && current.c == map.goalC && current.targetAngka == map.totalAngka) {
                return current;
            }

            for (int i = 0; i < 4; i++) {
                StateNode nodeBaru = IceSlidingLogic.slideMove(map.grid, map.costMatrix, current, i);
                if (nodeBaru != null && !(nodeBaru.isGameOver)) {
                    q.add(nodeBaru);
                }
            }
        }
        return null;
    }
}

```

Gambar 2. Algoritma BFS

```

public class DFS {
    private int iterationCount;

    public StateNode search(GameMap map) {
        iterationCount = 0;
        Stack<StateNode> stack = new Stack<>();
        boolean[][][] visited = new boolean[map.rows][map.cols][map.totalAngka + 1];

        StateNode startNode = new StateNode(map.startR, map.startC, totalCost: 0, targetAngka: 0, isGameOver: false,
        stack.push(startNode);

        while (!stack.isEmpty()) {
            StateNode current = stack.pop();

            if (visited[current.r][current.c][current.targetAngka]) {
                continue;
            }

            iterationCount++;
            visited[current.r][current.c][current.targetAngka] = true;

            if (current.r == map.goalR && current.c == map.goalC && current.targetAngka == map.totalAngka) {
                return current;
            }

            for (int i = 0; i < 4; i++) {
                StateNode nodeBaru = IceSlidingLogic.slideMove(map.grid, map.costMatrix, current, i);
                if (nodeBaru != null && !(nodeBaru.isGameOver)) {
                    stack.push(nodeBaru);
                }
            }
        }
        return null;
    }
}

```

Gambar 3. Algoritma DFS

```

public class GBFS{

    public StateNode search(GameMap map, int optionHeuristic){
        iterationCount = 0;
        Heuristic heuristic = new Heuristic(map);
        PriorityQueue<StateNode> pq = new PriorityQueue<>();
        boolean[][][] visited = new boolean[map.rows][map.cols][map.totalAngka + 1];

        StateNode node = new StateNode(map.startR, map.startC, totalCost: 0, targetAngka: 0, isGameOver: false,
        node.estimatedTotalCost = heuristic.estimatedCost(node.r, node.c, node.targetAngka, optionHeuristic);
        pq.add(node);

        while (!pq.isEmpty()){
            StateNode current = pq.poll();

            if (visited[current.r][current.c][current.targetAngka]){
                continue;
            }

            iterationCount++;
            visited[current.r][current.c][current.targetAngka] = true;

            if (current.r == map.goalR && current.c == map.goalC && current.targetAngka == map.totalAngka) {
                return current;
            }

            for (int i=0;i<4;i++){
                StateNode newNode = IceSlidingLogic.slideMove(map.grid, map.costMatrix, current, i);

                if (newNode != null && !(newNode.isGameOver)){
                    // EstimationTotalCost pakai Heuristik
                    newNode.estimatedTotalCost = heuristic.estimatedCost(newNode.r, newNode.c, newNode.targetA
                    pq.add(newNode);
                }
            }
        }
    }
}

```

Gambar 4. Algoritma GBFS

```

public class UCS {
    private int iterationCount;

    public StateNode search(GameMap map) {
        iterationCount = 0;
        PriorityQueue<StateNode> pq = new PriorityQueue<>();
        boolean[][][] visited = new boolean[map.rows][map.cols][map.totalAngka + 1];

        StateNode startNode = new StateNode(map.startR, map.startC, totalCost: 0, targetAngka: 0, isGameOver: false, ...);
        pq.add(startNode);

        while(!pq.isEmpty()) {
            StateNode current = pq.poll();

            if (visited[current.r][current.c][current.targetAngka]) {
                continue;
            }

            iterationCount++;
            visited[current.r][current.c][current.targetAngka] = true;

            if (current.r == map.goalR && current.c == map.goalC && current.targetAngka == map.totalAngka) {
                return current;
            }

            for (int i = 0; i < 4; i++) {
                StateNode nodeBaru = IceSlidingLogic.slideMove(map.grid, map.costMatrix, current, i);
                if (nodeBaru != null && !(nodeBaru.isGameOver)) {
                    pq.add(nodeBaru);
                }
            }
        }
        return null;
    }
}

```

Gambar 5. Algoritma UCS

```

public class Heuristic{
    // [1] Straight Line Distance (ke Target Goal/waypoint)
    private int SLD(int row, int col, int trow, int tcol){
        double x = Math.pow(col-tcol,2);
        double y = Math.pow(row-trow,2);
        int total = (int) Math.sqrt(x+y);
        return total;
    }

    // [2] Manhattan distance (ke Target Goal/waypoint)
    private int MD(int row, int col, int trow, int tcol){
        int x = Math.abs(col-tcol);
        int y = Math.abs(row-trow);
        int total = x+y;
        return total;
    }

    // [3] Manhattan Distance (Terhadap semua waypoint dan target)
    private int MDmulti(int row, int col, int targetAngka){
        int total = 0, r=row, c=col;
        for (int i=targetAngka;i<game.totalAngka;i++){
            int rt = game.waypointLocation[i][0];
            int ct = game.waypointLocation[i][1];
            total += MD(r,c,rt,ct);
            r = rt;
            c = ct;
        }
        total += MD(r,c,game.goalR,game.goalC);
        return total;
    }
}

```

Gambar 6. Heuristik-heuristik

```

public class Main {
    public static void main(String[] args) {
        // 4. === ALGORITMA ===
        StateNode solusi;
        int iterationCount;
        switch (algoritma) {
            case "1":
                System.out.println("Mencari jalan pakai UCS...");
                UCS ucs = new UCS();
                solusi = ucs.search(map);
                iterationCount = ucs.getIterationCount();
                break;
            case "2":
                System.out.println("Mencari jalan pakai GBFS...");
                GBFS gbfs = new GBFS();
                solusi = gbfs.search(map, optionHeuristic);
                iterationCount = gbfs.getIterationCount();
                break;
            case "3":
                System.out.println("Mencari jalan pakai A*...");
                Astar astar = new Astar();
                solusi = astar.search(map, optionHeuristic);
                iterationCount = astar.getIterationCount();
                break;
            case "4":
                System.out.println("Mencari jalan pakai BFS...");
                BFS bfs = new BFS();
                solusi = bfs.search(map);
                iterationCount = bfs.getIterationCount();
                break;
            case "5":
                System.out.println("Mencari jalan pakai DFS...");
                DFS dfs = new DFS();
                solusi = dfs.search(map);
                iterationCount = dfs.getIterationCount();
                break;
        }
    }
}

```

Gambar 7. Pemilihan Algoritma Pathfinding di dalam main

II Hasil Percobaan dan Analisis

Semua percobaan dan hasil percobaan ada di dalam folder **test**.

1. Algoritma UCS - test_1.txt

```

5 5
XXXXX
XZ0XX
XX*XX
XX0XX
XXXXX
999 999 999 999 999
999 1 2 999 999
999 999 3 999 999
999 999 4 999 999
999 999 999 999 999

```

```

Mencari jalan pakai UCS...
>> SOLUSI DITEMUKAN! <<
[1] Solusi Yang Ditemukan: RD
[2] Cost dari Solusi: 9
[3] Waktu eksekusi: 4 ms

[4] Banyak iterasi yang ditinjau: 4 iterasi

===== VISUALISASI SOLUSI =====
Legenda: Z = posisi aktor, . = lintasan slide pada langkah saat ini

Step 0 (posisi awal)
Path sementara : -
Total cost      : 0
Target berikut : 0
XXXXX
XZ0XX
XX*XX
XX0XX
XXXXX

Step 1 - gerak R (kanan)
Path sementara : R
Total cost      : 2
Target berikut : 0
XXXXX
X*ZXX
XX*XX
XX0XX
XXXXX

Step 2 - gerak D (bawah)
Path sementara : RD
Total cost      : 9
Target berikut : selesai
XXXXX
X*0XX
XX.XX
XXZXX
XXXXX

```

2. Algoritma GBFS (Heuristik Ke-3) - test_2.txt

```

5 8
XXXXXXXXX
XZ0***X
X*****1X
X0*****X
XXXXXXXXX
999 999 999 999 999 999 999
999 1 2 3 4 5 6 999
999 4 1 5 2 6 7 999
999 8 3 2 9 1 4 999
999 999 999 999 999 999 999

```

```

3. Memastikan Distance (ke semua target selanjutnya)
>> Masukkan Heuristik yang diinginkan (nomor): 3
WAKTUNYA BERMAIN!!!
Mencari jalan pakai GBFS...
>> SOLUSI DITEMUKAN! <<
[1] Solusi Yang Ditemukan: RDL
[2] Cost dari Solusi: 54
[3] Waktu eksekusi: 5 ms

[4] Banyak iterasi yang ditinjau: 4 iterasi

===== VISUALISASI SOLUSI =====
Legenda: Z = posisi aktor, . = lintasan slide pada langkah saat ini

Step 0 (posisi awal)
Path sementara : -
Total cost      : 0
Target berikut  : 0
XXXXXXXXX
XZ0***X
X****X
X***1X
X0***X
XXXXXXXXX

Step 1 - gerak R (kanan)
Path sementara : R
Total cost      : 20
Target berikut  : 1
XXXXXXXXX
X*0...ZX
X****X
X***1X
X0***X
XXXXXXXXX

Step 2 - gerak D (bawah)
Path sementara : RD
Total cost      : 31
Target berikut  : 0
XXXXXXXXX
X*0***X
X****X
X***1X
X0***ZX
XXXXXXXXX

Step 3 - gerak L (kiri)
Path sementara : RDL
Total cost      : 54
Target berikut  : selesai
XXXXXXXXX
X*0***X
X****X
XZ...*X
XXXXXXXXX

```

3. Algoritma A* (Heuristik ke-1) - test_3.txt

```

7 5
XXXXX
XZ0*X
X***X
X**1X
X***X
X0**X
XXXXX
999 999 999 999 999
999 2   3   4   999
999 5   1   6   999
999 3   2   5   999
999 7   4   1   999
999 6   8   9   999
999 999 999 999 999

```

```

>> Masukkan Heuristik yang diinginkan (nomor): 1
WAKTUNYA BERMAIN!!!
Mencari jalan pakai A*...
>> SOLUSI DITEMUKAN! <<
[1] Solusi Yang Ditemukan: RDL
[2] Cost dari Solusi: 42
[3] Waktu eksekusi: 6 ms

[4] Banyak iterasi yang ditinjau: 8 iterasi

===== VISUALISASI SOLUSI =====
Legenda: Z = posisi aktor, . = lintasan slide pada

Step 0 (posisi awal)
Path sementara : -
Total cost      : 0
Target berikut : 0
XXXXX
XZ0*X
X***X
X**1X
X***X
X0**X
XXXXX

Step 1 - gerak R (kanan)
Path sementara : R
Total cost      : 7
Target berikut : 1
XXXXX
X*0ZX
X***X
X**1X
X***X
X0**X
XXXXX

Step 2 - gerak D (bawah)
Path sementara : RD
Total cost      : 28
Target berikut : 0
XXXXX
X*0*X
X**.*X
X**1X
X**.*X
X0*ZX
XXXXX

Step 3 - gerak L (kiri)
Path sementara : RDL
Total cost      : 42
Target berikut : selesai
XXXXX
X*0*X

```

4. Algoritma GBFS (Heuristik ke-2) - test_4.txt

```

8 8
XXXXXXXXX
XZ0***X
X*****1X
X*****X
X*****2X
X**3***X
X0***X
XXXXXXXXX
999 999 999 999 999 999 999 999
999 1 2 3 4 5 6 999
999 3 8 2 5 1 4 999
999 6 2 7 4 3 9 999
999 5 9 1 6 8 2 999
999 7 4 3 2 5 1 999
999 2 6 8 9 3 4 999
999 999 999 999 999 999 999

```

```

Step 1 - gerak R (kanan)
Path sementara : R
Total cost      : 20
Target berikut  : 1
XXXXXXXXXX
X*0...ZX
X****1X
X*****X
X****2X
X**3***X
X0***XX
XXXXXXXXXX

Step 2 - gerak D (bawah)
Path sementara : RD
Total cost      : 36
Target berikut  : 3
XXXXXXXXXX
X*0****X
X****1X
X*****X
X****2X
X**3**ZX
X0***XX
XXXXXXXXXX

Step 3 - gerak L (kiri)
Path sementara : RDL
Total cost      : 57
Target berikut  : 0
XXXXXXXXXX
X*0****X
X****1X
X*****X
X****2X
XZ.3..*X
X0***XX
XXXXXXXXXX

Step 4 - gerak D (bawah)
Path sementara : RDLD
Total cost      : 59
Target berikut  : selesai
XXXXXXXXXX
X*0****X
X****1X
X*****X
X****2X
X**3***X
XZ***XX
XXXXXXXXXX

```

5. Algoritma A* (Heuristik ke-3) - test_5.txt

```

10 10
XXXXXXXXXX
XZ0*****X
X*****1X
X***7***X
X*****2X
X**4****X
X*****3X
X*****6*X
X0***5***X
XXXXXXXXXX
999 999 999 999 999 999 999 999 999 999
999 1 2 3 4 5 6 7 8 999
999 4 1 5 2 8 3 6 9 999
999 7 3 2 6 4 1 5 8 999
999 2 9 4 1 7 5 3 6 999
999 8 5 1 9 3 6 2 4 999
999 3 6 8 5 2 9 4 7 999
999 5 2 7 3 1 4 8 6 999
999 6 4 9 8 5 7 1 2 999
999 999 999 999 999 999 999 999 999

```



```

>> Masukkan Algoritma yang diinginkan (nomor): 3
Pilihan Heuristik:
1. SLD (Straight Line Distance)
2. Manhattan Distance (ke satu target selanjutnya)
3. Manhattan Distance (ke semua target selanjutnya)
>> Masukkan Heuristik yang diinginkan (nomor): 3
WAKTUNYA BERMAIN!!!
Mencari jalan pakai A*...
>> SOLUSI DITEMUKAN! <<
[1] Solusi Yang Ditemukan: RDLULD
[2] Cost dari Solusi: 169
[3] Waktu eksekusi: 6 ms

[4] Banyak iterasi yang ditinjau: 14 iterasi

===== VISUALISASI SOLUSI =====
Legenda: Z = posisi aktor, . = lintasan slide pada la

Step 0 (posisi awal)
Path sementara : -
Total cost      : 0
Target berikut : 0
XXXXXXXXXX
XZ0*****X
X*****1X
X*****X
X*****2X
X*****X
X*****3X
X*****X
X0*X****4X
XXXXXXXXXX

Step 1 - gerak R (kanan)
Path sementara : R
Total cost      : 35
Target berikut : 1
XXXXXXXXXX
X*0.....ZX
X*****1X
X*****X
X*****2X
X*****X
X*****3X
X*****X
X0*X****4X
XXXXXXXXXX

Step 2 - gerak D (bawah)
Path sementara : RD
Total cost      : 77
Target berikut : 0
XXXXXXXXXX

```

6. Algoritma BFS dan DFS (test.txt)

```

7 7
XXXXXX
X0***X
X**X**X
X***0X
X1***LX
XZ**X*X
XXXXXX
999 999 999 999 999 999 999
999 3 5 2 8 1 999
999 7 4 999 6 9 999
999 2 8 3 5 4 999
999 6 1 7 2 999 999
999 9 3 4 999 8 999
999 999 999 999 999 999 999

```

```

WAKTUNYA BERMAIN!!!
Mencari jalan pakai BFS...
>> SOLUSI DITEMUKAN! <<
[1] Solusi Yang Ditemukan: RULUDRUR
[2] Cost dari Solusi: 87
[3] Waktu eksekusi: 4 ms

[4] Banyak iterasi yang ditinjau: 14 iterasi

===== VISUALISASI SOLUSI =====
Legenda: Z = posisi aktor, . = lintasan slide pada
Step 0 (posisi awal)
Path sementara : -
Total cost      : 0
Target berikut  : 0
XXXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXXX

```

```

5. DFS
>> Masukkan Algoritma yang diinginkan (nomor): 5
WAKTUNYA BERMAIN!!!
Mencari jalan pakai DFS...
>> SOLUSI DITEMUKAN! <<
[1] Solusi Yang Ditemukan: RULRULDRULR
[2] Cost dari Solusi: 146
[3] Waktu eksekusi: 4 ms

[4] Banyak iterasi yang ditinjau: 12 iterasi

===== VISUALISASI SOLUSI =====
Legenda: Z = posisi aktor, . = lintasan slide pada
Step 0 (posisi awal)
Path sementara : -
Total cost      : 0
Target berikut  : 0
XXXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXXX

```

Analisis Hasil Percobaan

Semua percobaan menghasilkan solusi yang valid dan betul selagi test case yang diberikan mempunyai solusi. Pada percobaan yang tidak dimasukkan, program akan mengembalikan error apabila peta tidak berbentuk persegi panjang dan mengembalikan solusi tidak ada apabila test case tidak memberikan peta yang bisa diselesaikan.

Percobaan dengan ketiga heuristik dan kelima algoritma juga menghasilkan keberhasilan dalam menemukan jalan menuju solusi. Dapat diperhatikan juga bahwa cost dari masing-masing algoritma bisa berbeda dibuktikan dengan cost yang berbeda antara BFS dan DFS pada test case yang sama. Percobaan-percobaan di atas membuktikan bahwa semua heuristik dapat menemukan solusi yang tepat dan algoritma-algoritma yang diimplementasikan dapat berjalan dengan lancar dalam menemukan jalan menuju solusi. Hal ini terjadi karena setiap algoritma memiliki strategi eksplorasi yang berbeda, yaitu UCS memprioritaskan cost terkecil, GBFS memprioritaskan estimasi heuristik, A* menggabungkan cost aktual dan heuristik, sedangkan BFS dan DFS mengikuti urutan eksplorasi masing-masing. Dengan demikian, percobaan menunjukkan bahwa algoritma dan heuristik yang diimplementasikan dapat berjalan dengan baik, meskipun tidak semua algoritma menjamin cost solusi paling optimal.

III Lampiran

Pranala menuju *repository github*: https://github.com/Rmadhian/Tucil3_13524126_13524146

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)		✓
6	Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)	✓	
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)	✓	

Referensi:

- <https://mindyourdecisions.com/blog/2016/07/03/distance-between-two-random-points-in-a-square-sunday-puzzle/>
- <https://www.geeksforgeeks.org/data-science/manhattan-distance/>
- <https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/stima25-26.htm>